

AI Usage Note — .NET 8 AI Migration Assistant

This document summarizes the role of AI (Gemini LLM and development subagents) in the design, development, and debugging of the .NET Migration Assistant.

What AI Helped With

- **Static Regex Design:** Generated regex for parsing namespaces, classes, using statements, and legacy WebForms directives (such as matching Inherits attribute in .aspx, .svc, and .asmx files).
- **Architecture Skeletons:** Provided skeletons for ScannerEngine (dependency mapping), RiskEngine (blockers & rules), and MigrationEngine (backup & rollback).
- **UI Glassmorphic Design System:** Authored custom dark-themed CSS styling rules for the Streamlit dashboard, creating cards, background borders, and hover elevations.
- **Mock Suggestion Generation:** Designed offline fallback translation templates in client.py when API quotas are exhausted or keys are missing.
- **PDF Generation Layer:** Developed the ReportLab implementation, resolving table column layouts, typography configurations, and page alignments.
- **Single File Scan & Tracing:** Engineered CLI project root search routines (finding .csproj or .git) to run targeted scans on individual file paths.

What AI Got Wrong & Debugging Iterations

- **JSON Format Violations:** AI often wrapped JSON in markdown code blocks, causing parser errors. Remedied by setting response_mime_type to application/json and adding type-normalization filters.
- **MSBuild Duplicate Classes (Backup Bug):** Saving backups inside subfolders caused MSBuild to compile duplicates. Fixed by redirecting backups to the workspace root directory (backups/) outside compiler scope.
- **Mock Suggestion Precedence:** Applied general controller mock suggestions to Global.asax.cs. Remediated by re-ordering conditional checks to evaluate .asax file paths first.
- **Windows Encoding Crashes:** Terminal stdout/stderr encoding exceptions on Unicode arrows. Fixed by configuring sys.stdout to use utf-8 at the entrypoint.
- **API Rate Limit Quotas:** High-concurrency calls caused HTTP 429s. Capped parallel workers and added backoff delays (sleeping 35s on ResourceExhausted).

Best Prompts Used

A. Principal Architect Transformation Prompt

You are a Principal .NET Architect... Analyze the legacy source file and generate a precise, production-quality migration plan to port it to .NET 8.

Source Info

- Path: {file_path}
- Tech: {technology}
- Findings: {findings_str}

Instructions

1. Complete, compilable .NET 8 equivalent.
2. Beautifully formatted, using multiline structures and newlines.
3. Preserve all business logic.
4. Return ONLY a JSON object with schema: {summary, migration_strategy, unsupported_apis, dotnet8_equivalent, code_diff_markdown, confidence_score}.

B. Offline C# Mock Blueprint Prompt

Generate a modern C# Blazor Server equivalent (.razor) that maps WebForms event logic, ViewState bindings, and GridView layouts into a clean razor page utilizing MudBlazor component logic, injecting ApplicationDbContext and IHttpContextAccessor.

C. Streamlit Glassmorphic Styling Prompt

Enhance the Streamlit application UI (src/ui/app.py) to match a premium dark-themed design system. Write custom CSS to: 1. Style the default Deploy button with a green gradient. 2. Style HTML diff navigation links as cyan pill buttons. 3. Overhaul legends table to glassmorphic container with badges. 4. Standardize inputs/tabs/buttons to use glassmorphic borders and glows.

D. Single-File Search Prompt

Update the CLI scanner components to support scanning a single file. If a file path is provided, the CLI should trace upwards to find the closest project root directory (containing .csproj, packages.config, or .git), compute the relative path of the file, scan only that target file, and ensure backups and reports are successfully generated in the project root. Keep the folder scan backward-compatible.